

实验四：AnythingLLM 文档仓库查询

实验四：AnythingLLM 文档仓库查询

实验目的

1. 了解 AnythingLLM 的基本概念和使用方法
2. 掌握文档仓库查询的实现方式
3. 学习流式响应处理技术
4. 实现多轮对话中的文档查询集成

实验内容

实验过程

1. 代码结构分析 本实验包含两个 Python 文件：

chat_client.py - 主客户端 - **anythingllm_query()**: 查询 AnythingLLM 文档仓库 - **stream_llm_response()**: 流式获取 LLM 响应 - **summarize_chat_history()**: 聊天历史总结 - **main()**: 主函数，集成所有功能

test_anythingllm.py - 测试脚本 - 测试环境变量加载 - 测试 AnythingLLM 查询功能

2. AnythingLLM 查询实现 API 调用配置

```
def anythingllm_query(query):
    env_vars = load_env()
    api_key = env_vars.get('ANYTHINGLLM_API_KEY', '')
    workspace_slug = env_vars.get('ANYTHING_WORKSPACE_SLUG', 'project')

    api_url = f"http://localhost:3001/api/v1/workspace/{workspace_slug}/chat"

    request_body = json.dumps({
        "message": query
    }, ensure_ascii=False)
```

```

curl_cmd = [
    'curl', '-X', 'POST',
    api_url,
    '-H', 'Content-Type: application/json; charset=utf-8',
    '-H', f'Authorization: Bearer {api_key}',
    '-d', request_body
]

```

响应解析

```

if 'textResponse' in response_data:
    return response_data['textResponse']
elif 'sources' in response_data:
    response_text = response_data.get('textResponse', '')
    sources = response_data.get('sources', [])
    if sources:
        sources_text = "\n\n参考来源: \n"
        for i, source in enumerate(sources, 1):
            title = source.get('title', '未知文档')
            sources_text += f"{i}. {title}\n"
        return response_text + sources_text

```

3. 流式响应处理 流式输出实现

```

def stream_llm_response(base_url, model, api_key, messages, max_tokens=500):
    data = {
        "model": model,
        "messages": messages,
        "max_tokens": max_tokens,
        "stream": True
    }

    # 发送请求并逐行读取响应
    while True:
        line = response.readline().decode('utf-8')
        if not line:
            break
        line = line.strip()
        if line.startswith('data: '):
            data_str = line[6:]
            if data_str == '[DONE]':
                break
            try:
                chunk = json.loads(data_str)

```

```
        if 'choices' in chunk and len(chunk['choices']) > 0:
            delta = chunk['choices'][0].get('delta', {})
            content = delta.get('content', '')
            if content:
                print(content, end='', flush=True)
                full_content += content
    except:
        continue
```

4. 关键词触发机制 文档查询触发

```
warehouse_keywords = ["文档仓库", "文件仓库", "仓库"]
if any(keyword in text_lower for keyword in warehouse_keywords):
    print("\n【系统】检测到文档仓库查询，正在执行...")
    query = user_input
    for keyword in warehouse_keywords:
        if keyword in query:
            query = query.replace(keyword, '').strip()
    result = call_function("anythingllm_query", {"query": query})
```

实验结果

功能验证

功能	chat_client.py	test_anythingllm.py
AnythingLLM 查询	正确	正确
流式响应输出	正确	错误
聊天历史压缩	正确	错误
环境变量加载测试	错误	正确

输出示例

```
=====
AI 智能体（支持AnythingLLM文档仓库查询）
=====
提示：
- 输入消息与AI对话
- 提到'文档仓库'、'文件仓库'、'仓库'时会触发文档查询
- 聊天超过5轮或上下文超过3k时会自动压缩
- 输入 'quit' 或 'exit' 退出
=====
连接到: http://127.0.0.1:1234/v1
```

使用模型：qwen/qwen3.5-2b

=====

你：文档仓库中关于项目介绍的内容

【系统】检测到文档仓库查询，正在执行...

【查询结果】

本项目是一个AI智能助手系统，主要功能包括：

1. 文档查询与检索
2. 自然语言对话
3. 工具调用能力
4. 聊天历史管理

参考来源：

1. 项目说明文档.md

你：这个系统有什么特点？

AI：本系统具有以下特点：

1. 支持多模态交互
2. 具备工具调用能力
3. 支持长对话管理

[耗时：1.25秒]

实验总结

实验成果

1. 成功集成 AnythingLLM 文档仓库查询功能
2. 实现流式响应输出，提升用户体验
3. 实现关键词自动触发文档查询
4. 实现聊天历史压缩功能

技术要点

1. **API 集成**: 通过 curl 调用 AnythingLLM API，处理中文编码
2. **流式响应**: 使用 Server-Sent Events 实现实时响应输出
3. **关键词触发**: 通过关键词检测自动触发特定功能
4. **上下文管理**: 智能压缩聊天历史，控制 Token 消耗

改进建议

1. 可添加文档查询结果缓存机制

2. 可实现多仓库查询支持
3. 可添加查询历史记录
4. 可实现查询结果的语义分析和推荐